

AI Assisted Coding

Name :M.sai charani

Hall : 2303a52192

Batch : 34

Lab 3: Prompt Engineering – Improving Prompts and Context Management

Task Description-1

- Progressive Prompting for Calculator Design: Ask the AI to design a simple calculator program by initially providing only the function name. Gradually enhance the prompt by adding comments and usage examples.

The screenshot shows a VS Code editor with a Python file named `calculator.py`. The code defines a `calculator` function that takes two numbers and an operator as input. It includes error handling for invalid operators and non-numeric inputs. The function performs addition, subtraction, multiplication, and division. The terminal shows the execution of the program, where the user enters two numbers (10 and 5) and an operator (+), resulting in the output 15. The right sidebar displays the AI's response to the prompt, showing the code generation process and the final output.

```
def calculator(a=None, b=None, op=None):
    if op is None:
        print("Invalid operator. Choose one of: +, -, *, /")
        return None
    if a is None or b is None:
        print("Error: Invalid numeric input.")
        return None
    if op == "+":
        result = a + b
    elif op == "-":
        result = a - b
    elif op == "*":
        result = a * b
    elif op == "/":
        result = a / b
```

Terminal Output:

```
PS C:\Users\MSAI> python calculator.py
Enter the first number: 10
Enter the second number: 5
Enter an operator (+, -, *, /): +
Result: 10.0 + 5.0 = 15.0
```

Task Description-2

- Refining Prompts for Sorting Logic: Start with a vague prompt for sorting student marks, then refine it to clearly specify sorting order and constraints.


```
def grade_from_percentage(pct):
    """Return grade letter for percentage using given mapping.

    A: >=90, B: >=75, C: >=60, D: <60
    """
    try:
        pct = float(pct)
    except Exception:
        return 'Invalid'
    if pct >= 90:
        return 'A'
    if pct >= 75:
        return 'B'
    if pct >= 60:
        return 'C'
    return 'D'

def grade_student(name, marks):
    """Compute total, percentage, and grade for a student.

    'marks' may be a dict mapping subject->mark or an iterable of numeric marks.
    Returns a dict with 'name', 'marks' (as dict), 'total', 'percentage', 'grade'.
    Percentage is calculated as the average mark (treating each subject as out of 100).
    """
    # Normalize marks into an ordered dict-like mapping
    if isinstance(marks, dict):
        subj_marks = dict(marks)
    else:
        # assume iterable of marks without subject names
        subj_marks = {'Subject (i-1)': float(n) for i, n in enumerate(marks)}

    # Initialize numeric marks and compute total
```

```
def grade_student(name, marks):
    # Initialize numeric marks and compute total
    total = 0.0
    count = 0
    for k, v in list(subj_marks.items()):
        try:
            mv = float(v)
        except Exception:
            mv = 0.0
        subj_marks[k] = mv
        total += mv
        count += 1

    if count == 0:
        percentage = 0.0
    else:
        average = total / count
        percentage = average # treating average as percentage (out of 100)

    grade = grade_from_percentage(percentage)
    return {
        'name': name,
        'marks': subj_marks,
        'total': total,
        'percentage': round(percentage, 2),
        'grade': grade,
    }

def interactive_grading_system():
    """Interactively accept multiple students and their subject marks.

    For each student:
```

```

101 def interactive_grading_system():
102     """Interactively accept multiple students and their subject marks.
103
104     For each student:
105     - Prompt for student name (blank to finish)
106     - Prompt repeatedly for subject name and mark (blank subject to finish)
107     - Validate numeric marks
108     - Compute and display total, percentage, and grade in formatted output
109     """
110     results = []
111     print("Enter student data. Leave student name blank to finish.")
112     while True:
113         name = input("Enter student name: ").strip()
114         if name == "":
115             break
116
117         subjects = {}
118         print("Enter subject name and mark. Leave subject blank to finish.")
119         while True:
120             subj = input("Subject name: ").strip()
121             if subj == "":
122                 break
123             mark_raw = input("Mark (0-100): ").strip()
124             try:
125                 mark = float(mark_raw)
126             except ValueError:
127                 print("Invalid mark - please enter a numeric value.")
128                 continue
129             subjects[subj] = mark
130
131         rec = grade_student(name, subjects)
132         results.append(rec)

```

```

133
134     # Display results
135     print("\n--- Results ---")
136     print("Student: {name}")
137     print("Subjects: {subjects}")
138     for s, m in rec["marks"].items():
139         print(f"    {s}: {m}")
140     total_possible = len(rec["marks"]) * 100 if rec["marks"] else 0
141     print(f"Total: {rec['total']} / {total_possible}")
142     print(f"Percent: {rec['percentage']}")
143     print(f"Grade: {rec['grade']}")
144     print("-----")
145
146     again = input("Add another student? (y/n): ").strip().lower()
147     if again not in ("y", "n"):
148         break
149
150     except KeyboardInterrupt:
151         print("\nInterrupted - returning to caller.")
152
153     return results
154
155 if __name__ == "__main__":
156     # Run the interactive grading system when executed directly.
157     interactive_grading_system()

```

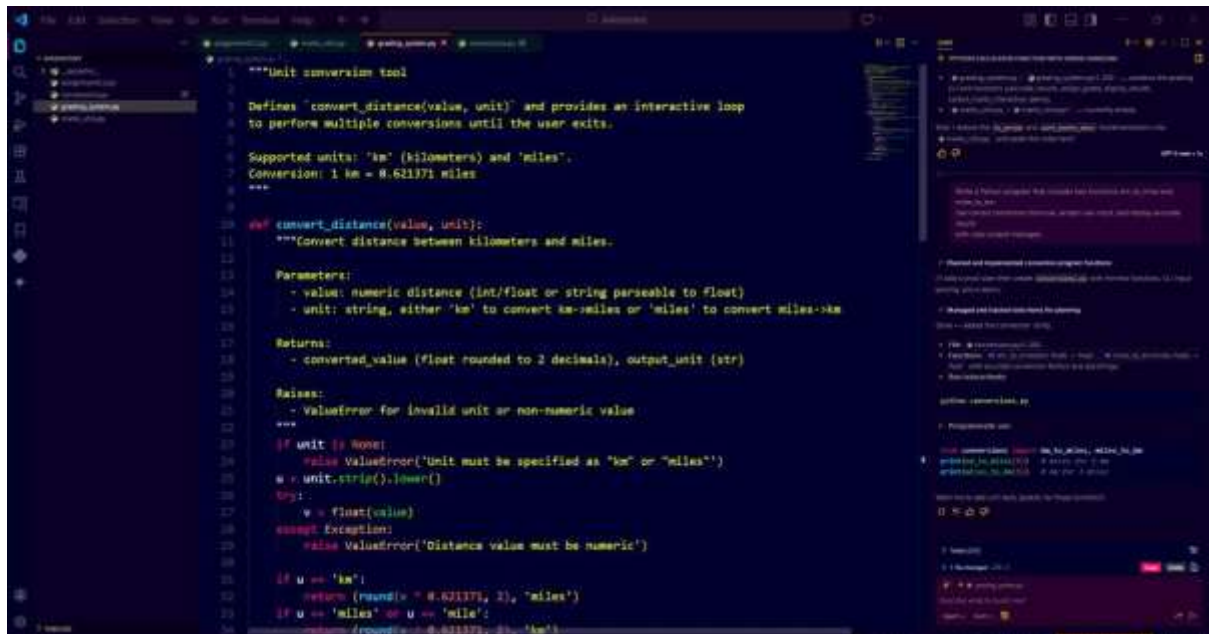
```

158
159 # Run the interactive grading system when executed directly.
160 if __name__ == "__main__":
161     interactive_grading_system()

```


Task Description-5

- Analyzing Prompt Specificity in Unit Conversion Functions: Improving a Unit Conversion Function (Kilometers to Miles and Miles to Kilometers) Using Clear Instructions.



The screenshot shows a code editor with a Python file named `unit_converter.py`. The code defines a function `convert_distance(value, unit)` that converts distances between kilometers and miles. The function includes docstrings, parameter descriptions, and error handling. The right sidebar shows a file explorer and a search bar.

```
"""Unit conversion tool

Defines "convert_distance(value, unit)" and provides an interactive loop
to perform multiple conversions until the user exits.

Supported units: 'km' (kilometers) and 'miles'.
Conversion: 1 km = 0.621371 miles
"""

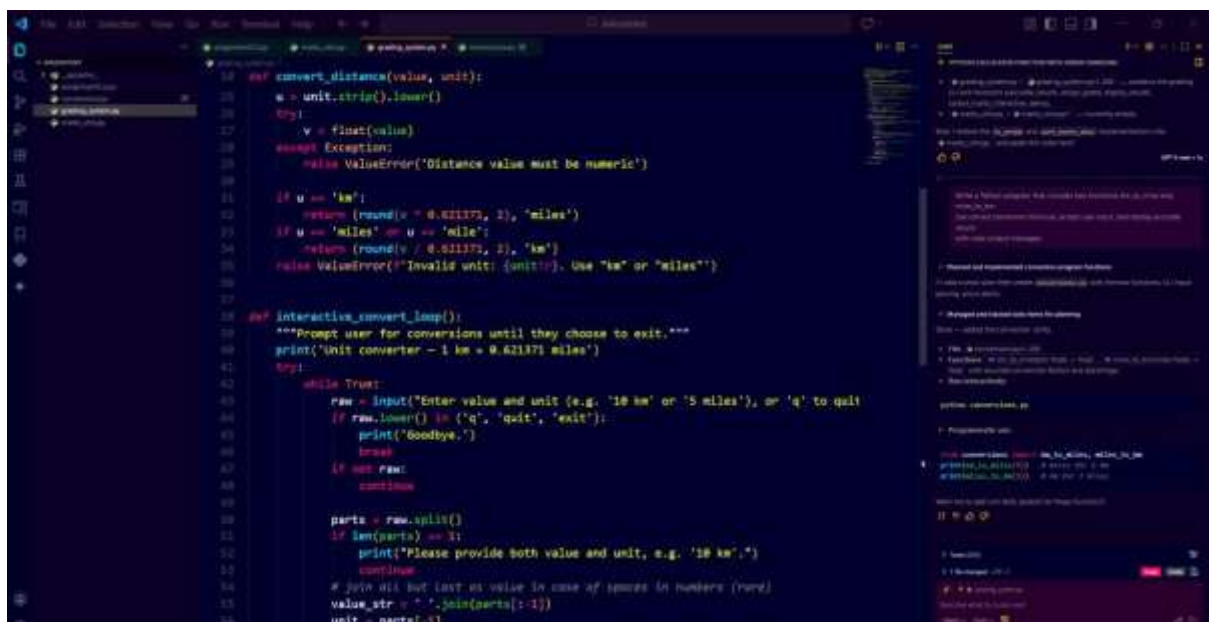
def convert_distance(value, unit):
    """Convert distance between kilometers and miles.

    Parameters:
    - value: numeric distance (int/float or string parseable to float)
    - unit: string, either 'km' to convert km->miles or 'miles' to convert miles->km

    Returns:
    - converted_value (float rounded to 2 decimals), output_unit (str)

    Raises:
    - ValueError for invalid unit or non-numeric value
    """
    if unit is None:
        raise ValueError('Unit must be specified as "km" or "miles"')
    u = unit.strip().lower()
    try:
        v = float(value)
    except Exception:
        raise ValueError('Distance value must be numeric')

    if u == 'km':
        return (round(v * 0.621371, 2), 'miles')
    if u == 'miles' or u == 'mile':
        return (round(v / 0.621371, 2), 'km')
    raise ValueError('Invalid unit: {unit}. Use "km" or "miles"')
```



The screenshot shows a code editor with a Python file named `unit_converter.py`. The code defines an interactive loop function `interactive_convert_loop()` that prompts the user for conversions until they choose to exit. The function includes docstrings, parameter descriptions, and error handling. The right sidebar shows a file explorer and a search bar.

```
def interactive_convert_loop():
    """Prompt user for conversions until they choose to exit."""
    print('Unit converter - 1 km = 0.621371 miles')
    try:
        while True:
            raw = input('Enter value and unit (e.g. "10 km" or "5 miles"), or "q" to quit\n')
            if raw.lower() in ('q', 'quit', 'exit'):
                print('Goodbye.')
                break
            if not raw:
                continue

            parts = raw.split()
            if len(parts) == 1:
                print('Please provide both value and unit, e.g. "10 km".')
                continue
            # Join all but last as value in case of spaces in numbers (rare)
            value_str = ' '.join(parts[:-1])
            unit = parts[-1]
```



```

1 def interactive_convert_loop():
2     # Get raw input
3     while True:
4         raw_input = input("Enter a distance value and unit, e.g. '10 km': ")
5         # Parse the input
6         parts = raw_input.split()
7         if len(parts) == 1:
8             print("Please provide both value and unit, e.g. '10 km'.")
9             continue
10        # Join all but last as value in case of spaces in numbers (rare)
11        value_str = ' '.join(parts[:-1])
12        unit = parts[-1]
13
14        # Try to convert
15        try:
16            converted, out_unit = convert_distance(value_str, unit)
17        except ValueError as e:
18            print("Error: ", e)
19            continue
20
21        # Format and display
22        try:
23            val_num = float(value_str)
24        except Exception:
25            val_num = value_str
26        print(f"{val_num} {unit} -> {converted} {out_unit}")
27
28    except KeyboardInterrupt:
29        print("\nInterrupted. Exiting.")
30
31    # Main loop
32    if __name__ == '__main__':
33        interactive_convert_loop()

```

The screenshot shows a Jupyter Notebook environment. The left sidebar contains a file explorer with a 'notebooks' folder. The main area displays a Python script for an interactive distance converter. The script uses a loop to repeatedly prompt the user for a distance value and unit. It parses the input, converts the distance to meters, and displays the result in both the original unit and meters. The right sidebar shows the Jupyter Notebook's output area, which is currently empty.